

# Web Services, Web API, REST, MVC

## SWE 4537: Server Programming Lecture 01.2

Asaduzzaman Herok  
Department of CSE  
Islamic University of Technology

13 November 2025

# Recap: Where We Left Off (Lecture 1)

## The Core Loop

In our last lecture, we established the fundamentals of web communication.

- 👉 A browser (**Client**) sends an **HTTP Request** to a server at a specific **IP** and **Port**.
- 👉 The **Server** then processes the request and sends back an **HTTP Response** (like 200 OK + HTML).

# The Bridge to Today

## Today's Focus

What is the actual **program** on the server that is responsible for:

- 👉 Listening for those requests?
- 👉 Deciding what to do with them?
- 👉 Creating that response?

## The Answer

This program, in its broadest sense, is a **Web Service**.

# Key Distinction: App vs. Service

## Web Application (For Humans)

A program whose primary "user" is a human, and its output is a **User Interface (UI)**.

👉 **Analogy:** A fully-assembled car.

👉 **Response:** Sends **HTML, CSS, and JavaScript** for a browser to render visually.

## Web Service (For Machines)

A program whose primary "user" is another program (a machine), and its output is **raw data**.

👉 **Analogy:** The engine from the car factory.

👉 **Response:** Sends **JSON** (or XML) for another program to parse and use.

# Paradigms of Web Services

How do machines talk to each other?

## RPC (Remote Procedure Call)

The oldest idea. "Just call a function on another server."

- 👉 Client code calls `getUsers()` or `calculateTotal(orderID)`.
- 👉 It's simple, but **brittle**. What if the server renames the function? How do you discover functions?

## SOAP (Simple Object Access Protocol)

A formal **protocol** (a strict set of rules) for web services.

- 👉 Uses complex **XML** for all messages.
- 👉 Has a rigid contract (WSDL) defining all functions.
- 👉 Very powerful (built-in security, transactions) but very **heavyweight** and complex.

# What is WSDL? (Part I)

## Web Services Description Language

### The "Contract"

An **XML-based** file that acts as a formal "contract" or "user manual" for a web service.

It tells a client application *exactly* how to structure a request and what to expect in response.

### WSDL vs. SOAP

👉 **WSDL** is the **description** (the "menu").

👉 **SOAP** is the **message format** (the "order").

You read the WSDL to learn how to build the SOAP message.

# What is WSDL? (Part II)

## Key Components:

A WSDL file answers these 5 questions:

- 👉 **<types>**: What are the data types?
- 👉 **<message>**: What do the payloads look like?
- 👉 **<portType>**: What are the abstract operations?
- 👉 **<binding>**: What protocol do we use? (e.g., SOAP)
- 👉 **<service>**: Where is the service's URL?

# What is WSDL? (Part III)

## Simplified WSDL Example

```
<!-- 1. The data types we will use --> <types> <schema> <element name="UserID" type="xs:int"/> <element  
name="UserName" type="xs:string"/> </schema> </types>  
<!-- 2. The request/response messages --> <message name="getUserRequest"> <part name="id"  
element="tns:UserID"/> </message> <message name="getUserResponse"> <part name="name"  
element="tns:UserName"/> </message>  
<!-- 3. The abstract operations (functions) --> <portType name="UserPortType"> <operation name="getUser">  
<input message="tns:getUserRequest"/> <output message="tns:getUserResponse"/> </operation> </portType>  
<!-- 4. The concrete protocol (SOAP) --> <binding name="UserBinding" type="tns:UserPortType"> <soap:binding  
style="rpc" ... /> <operation name="getUser"> <soap:operation soapAction="getUser"/> <input><soap:body  
.../></input> <output><soap:body .../></output> </operation> </binding>  
<!-- 5. The physical address (URL) --> <service name="UserService"> <port name="UserPort"  
binding="tns:UserBinding"> <soap:address location="http://api.example.com/UserService"/> </port> </service>  
</definitions>
```



# What is an API?

## API: Application Programming Interface

This is the "public-facing" part of the server. It's the **contract** that defines *how* other programs can talk to your service.

- 👉 It's the "waiter" for the kitchen.
- 👉 It's the "menu" that lists the rules.
- 👉 It's the technical specification manual.

# The Restaurant Analogy

- 👉 👤 **The Client (You):** You want food (data).
- 👉 🍴 **The Server (The Kitchen):** Has the data (ingredients) and the logic (chefs) to make your food. This is the **Web Service**.
- 👉 🧑 **The API (The Waiter):** You (the client) are not allowed to go into the kitchen. You must talk to the waiter.

# Connecting the Analogy to Theory

How do different paradigms use the "waiter"?

## RPC Analogy

You shout into the kitchen: `"getMeSteak()"`.

**Problem:** It's simple, but what if the chef's function is named `fetchSteak(temp)`? The call fails. It's brittle.

## SOAP Analogy

The waiter gives you a 100-page legal document (the WSDL) in Latin (XML). You must fill out Form 32-B *exactly* to order.

**Problem:** It's very precise, but extremely slow and difficult to work with.

## REST Analogy

The waiter gives you a simple, well-organized **menu** (the API endpoints). You use simple words (**Verbs**) to pick items (**Nouns**). "I'll GET the steak."

# What is REST?

## REST (Representational State Transfer)

An **architectural style**, not a protocol. It's a philosophy defined by a set of **constraints** that, if followed, lead to a scalable, evolvable system.

### The 6 Guiding Constraints

- 1 **Client-Server:** Enforces Separation of Concerns.
- 2 **Stateless:** (The most important one! More soon.)
- 3 **Cacheable:** Responses can be cached by the client.
- 4 **Uniform Interface:** (The key to REST!)
- 5 **Layered System:** Proxies/gateways can exist in the middle.
- 6 **Code on Demand (Optional):** Server can send code (e.g., JavaScript).

# REST vs. RPC: A Shift in Thinking

This is the core of the "Uniform Interface" constraint.

## RPC (Remote Procedure Call)

It's about **Actions (Verbs)**.

👉 Endpoints are functions: `/getAllTodos`, `/createNewTodo`.

👉 **Problem:** This is brittle. What if you need todos by date? You must add a *new function* and *new endpoint*: `/getAllTodosByDate`. The API explodes.

## REST (Representational State Transfer)

It's about **Resources (Nouns)**.

👉 You have one resource: `/todos`.

👉 You use **parameters** for filtering: `/todos?date=2025-11-13`.

👉 You use **HTTP Verbs** (the existing protocol!) for actions.

# The RESTful "Menu" (Part 1: The Collection)

## Resource: /todos

(Refers to the collection of ALL todos)

### 👉 GET /todos

- ▶ **Action:** Read
- ▶ **Meaning:** "Get me a list of all todos."
- ▶ **Note:** Data (like filters) is sent in the URL (e.g., /todos?user=1).

### 👉 POST /todos →

- ▶ **Action:** Create
- ▶ **Meaning:** "Create a new todo." (Data is in the body).
- ▶ **Note:** Data is in the **body**, not visible in the URL. Good for sensitive data.

# The RESTful "Menu" (Part 2: A Specific Item)

## Resource: /todos/123

(Refers to a single, specific todo)

- 👉 GET /todos/123 → **Read**: "Get details for todo #123."
- 👉 PUT /todos/123 → **Update**: "Replace todo #123."
- 👉 DELETE /todos/123 → **Delete**: "Delete todo #123."

## Note

This is clean, predictable, and easy to understand. We use the built-in power of HTTP instead of inventing new URLs.

# More HTTP Methods

## HEAD

Identical to GET, but it **does not return the message body**

👉 **Use:** Check if a resource exists or get its metadata (like Content-Length) without downloading the whole file.

## PATCH

Applies a **partial replacement** to a resource

👉 **Use:** Unlike PUT (which replaces everything), PATCH is for changing just one field (e.g., updating *only* a todo's title).

## OPTIONS

Asks the server to list the **supported HTTP methods** for a given URL

👉 **Use:** Used to describe the communication options for the target resource, such as the allowed HTTP methods.



# REST: The "Representation" Part

## The "Representation" in REST

The server doesn't send you the actual database object. It sends a *representation* of that resource.

👉 GET /todos/123 → **JSON** (for a mobile app)

👉 GET /todos/123 → **HTML** (for a website)

## Content Negotiation

This is formally called **Content Negotiation**.

👉 Client sends: Accept: application/json

👉 Server replies: Content-Type: application/json

For this course, we will *always* use JSON.

# REST's Most Important Constraint: Statelessness

## Stateless

The server does not store any client "session" state between requests.

### What this means:

Every single request from the client must contain **all** the information needed for the server to process it.

- 👉 **Bad (Stateful):** 1. POST /login 2. GET /my-todos (Server just "remembers" who you are).
- 👉 **Good (Stateless):** 1. POST /login (Server returns an `access_token`). 2. GET /todos (Client sends `Authorization: Bearer <token>` in the header).

## Why?

**Scalability.** Any server in a 1000-server cluster can handle any request, as no server needs to remember "who you are."

# The Problem: "The Big Messy File"

## The Naive Approach

"Okay, so we're going to build a program that listens for these requests. We'll use a framework like FastAPI."

## The Temptation

"In Week 2, our `main.py` file will look like this..."

# The "Messy" File Example

main.py

```
# main.py
@app.get("/todos")
def get_all_todos():
    # 1. Connect to the database...
    # 2. Write a SQL query...
    # 3. Format the data into JSON...
    # 4. Return the JSON...

@app.post("/todos")
def create_todo():
    # 1. Get the JSON from the request body...
    # 2. Connect to the database...
    # 3. Write a SQL 'INSERT' query...
    # 4. Return a '201 Created' status...
```

# The Problem: "The Big Messy File"

## The Problem

What happens when we add `/users`, `/posts`, `/comments`, and all their GET, POST, PUT, DELETE methods?

- 👉 Our `main.py` file becomes 5,000 lines long!
- 👉 All our database code, business logic, and API code are mixed together.
- 👉 It's impossible to test, debug, or maintain.

**This violates a core software principle:**

Separation of Concerns

# The Solution: Architectural Patterns

## Architectural Pattern

A template for organizing your code. A reusable solution to a common problem.

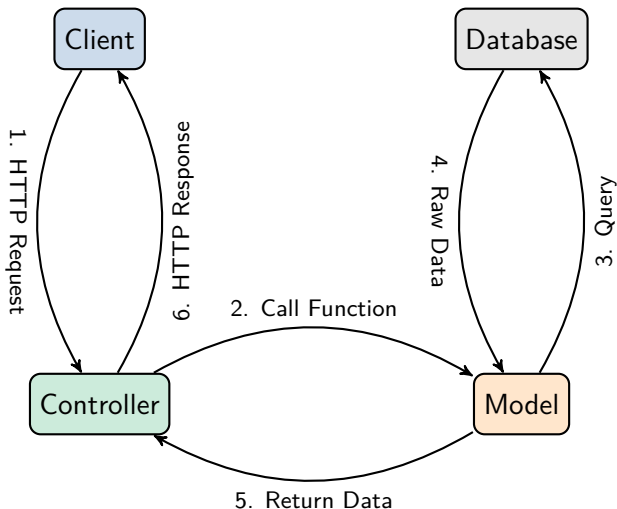
## Our Pattern: MVC

The most famous pattern is **MVC (Model-View-Controller)**. We will use a variation of this for our API.

## Core Idea

**Separation of Concerns.** We separate our code into three distinct jobs.

# The MVC Pattern (Our API Flow)



# MVC: The Components (M)

## M - Model (The "Data" Layer)

This is the "database" layer. It handles all data and **business logic**.

- 👉 Its *only* job is to talk to the database (e.g., `SELECT * from todos`).
- 👉 It contains all your SQL queries.
- 👉 It enforces rules (e.g., "password must be 8 characters").
- 👉 It **doesn't know the API exists**. It just returns data.



# MVC: The Components (C)

## C - Controller (The "Brain" / "Waiter")

This is the "brain" of the operation. It's the "traffic cop" that directs flow.

- 👉 It **handles the HTTP Request** (e.g., GET /todos).
- 👉 It calls the Model ("Hey Model, get me all the todos").
- 👉 The Model gives it raw data.
- 👉 The Controller then formats that data into a Response (the "View").

# MVC: The Components (V)

## V - View (The "Presentation" Layer)

Formats the data for the "user".

- 👉 **In a traditional website:** This is the **HTML file** / template.
- 👉 **For our API:** The "user" is a machine. The "View" is simply the **JSON** that the Controller sends back.

# Our New, Clean Flow (MVC)

This is the same process, but organized:

- 1 Request for GET /todos hits the **Controller**.
- 2 **Controller** calls the **Model** (`model.get_all_todos()`).
- 3 **Model** queries the Database and returns raw data.
- 4 **Controller** formats the data into JSON (the "View").
- 5 **Controller** sends the 200 OK Response.

## Why is this better?

If we change the database? **Only touch the Model.**

If we change the URL? **Only touch the Controller.**

# We Have All The Theory!

We now have all the theory we need.

- 👉 We know the rules (**HTTP**).
- 👉 We know the design (**REST**).
- 👉 We know the organization (**MVC**).

# Questions?

## Thank You!

Thank you for your time and attention.  
Any questions or clarifications?

